UniAgent: Integrating A2A, x402, and ERC-8004 for Autonomous Agent Marketplaces

Ben Trovato*
G.K.M. Tobin*
trovato@corporation.com
webmaster@marysville-ohio.com
Institute for Clarity in Documentation
Dublin, Ohio, USA

Abstract

Large language model (LLM) agents are increasingly able to act on behalf of users, but the infrastructure that lets them find, hire, and pay one another across organizational boundaries is still fragmented. Open protocols already exist for the individual pieces: the Agent2Agent (A2A) protocol standardizes agent-to-agent communication, x402 turns the HTTP 402 status code into a micropayment channel, and ERC-8004 records agent identity on chain. However, these protocols have so far been demonstrated only in isolation or as one-to-one proofs of concept, and no end-to-end service architecture ties them together so that a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. In this paper we present *UniAgent*, a decentralized agent marketplace that integrates A2A, x402, and ERC-8004 under an LLM-based orchestration layer. A three-layer design separates a *marketplace layer* (an on-chain ERC-8004 registry kept in sync with an off-chain cache through blockchain event webhooks), an *orchestration layer* (a ReAct agent that discovers, selects, and invokes external agents through two tools), and a *payment layer* (x402 with delegated EIP-3009 signatures and a server-side budget policy). We describe a working prototype deployed on the Base Sepolia testnet and report an end-to-end evaluation over flight- and hotel-booking agents. UniAgent shows that open communication, on-chain identity, and protocol-native micropayments can be combined into a coherent, vendor-neutral marketplace in which user tasks are carried out autonomously across independent providers.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Information systems** → *Web services*; • **Software and its engineering** → *Distributed systems organizing principles.*

Keywords

AI agents, agent marketplace, A2A protocol, x402, ERC-8004, micropayments, on-chain identity, LLM orchestration, ReAct

ACM Reference Format:
Ben Trovato and G.K.M. Tobin. 2026. UniAgent: Integrating A2A, x402, and ERC-8004 for Autonomous Agent Marketplaces. In *Proceedings of International Conference on Cloud Computing and Management (ICCCM 2026) (ICCCM '26).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large language models (LLMs) have made it practical to build autonomous agents that plan, call tools, and complete multi-step tasks on behalf of a user. As these agents proliferate, a natural next step is for one agent to delegate work to another: a travel assistant might hire a specialized flight agent and a separate hotel agent, each operated by a different organization. Realizing this vision requires three capabilities that cut across trust boundaries: agents must be able to *communicate* with one another, to be *discovered* and *identified*, and to *pay* for services without a shared account or a manual contract.

Open standards have recently emerged for each of these capabilities. The Agent2Agent (A2A) protocol [9] defines how agents describe themselves through an *Agent Card* and exchange messages over JSON-RPC. The x402 protocol [7] reuses the long-dormant HTTP 402 "Payment Required" status code to attach stablecoin micropayments to ordinary web requests. ERC-8004 [8] provides an on-chain registry of agent identities, building on the ERC-721 token standard so that any party can look up an agent without trusting a central directory. Individually, these protocols are compelling; together, they suggest a decentralized alternative to the closed agent stores offered by today's platforms.

Despite this progress, three gaps remain. **First**, A2A standardizes communication but explicitly leaves *discovery*—how a client finds an appropriate agent in the first place—outside its scope. **Second**, recent work by Vaziry et al. [10] demonstrated the technical feasibility of combining A2A, x402, and on-chain Agent Cards, but did so at the protocol level, as one-to-one interactions between a fixed pair of agents; it did not provide a service architecture that takes a user's high-level task and autonomously drives discovery, delegation, and payment to completion. **Third**, centralized platforms such as the OpenAI GPT Store [6] do offer end-to-end experiences, but they confine the agent ecosystem to plug-ins inside a single vendor's walled garden [3], with platform-specific registration, discovery, and billing.

*Both authors contributed equally to this research.

ICCCM '26, TBD

ACM ISBN 978-1-4503-XXXX-X/2026/XX
https://doi.org/XXXXXXX.XXXXXXX

The result is a missing middle: the individual protocols exist, and closed platforms show what a smooth user experience looks like, but no *open* architecture connects the protocols through an orchestration layer that can turn a natural-language request into cross-organizational agent transactions. This motivates our research question:

> *How can open agent communication, on-chain identity, and protocol-level micropayments be integrated into a cohesive marketplace architecture with LLM-based orchestration, so that a user's high-level task is autonomously decomposed into agent discovery, delegation, and payment across organizational boundaries?*

We answer this question with *UniAgent*, a decentralized agent marketplace whose architecture is summarized in Figure 1. UniAgent takes the registration, discovery, and payment functions that a centralized platform bundles together and re-implements them on open protocols—A2A and x402—and a standardized on-chain identity layer—ERC-8004. On top of these, an LLM ReAct agent [12] acts as an orchestration layer that analyzes the task, discovers candidate agents, selects among them, and executes them with automatic payment.

Contributions. This paper makes three contributions:

(1) **An integrated marketplace architecture** that unifies A2A, x402, and ERC-8004 behind an orchestration layer, organized as three loosely coupled layers (marketplace, orchestration, payment).
(2) **An autonomous LLM pipeline** in which a ReAct agent drives task analysis, agent discovery, selection, and payment through two tools (`discover_agents` and `execute_agent`), with a server-side budget policy that keeps spending under user control.
(3) **A working prototype and end-to-end demonstration** on the Base Sepolia testnet, exercising the full flow over external flight- and hotel-booking agents and reporting success rate, latency, and on-chain payment cost.

The rest of the paper is organized as follows. Section 2 reviews the protocols UniAgent builds on. Section 3 positions our work against related research, with particular attention to Vaziry et al. Section 4 presents the three-layer architecture, the core of our contribution. Section 5 describes the prototype, and Section 6 evaluates it. Section 7 discusses limitations and future work, and Section 8 concludes.

2 Background

2.1 A2A Protocol

The Agent2Agent (A2A) protocol [9] is an open standard for communication between independent agents. Each agent publishes an *Agent Card*, a JSON document served at a well-known URL (`/.well-known/agent.json`) that describes the agent's name, capabilities, service endpoint, and—in payment-aware deployments—its price. Clients then exchange task messages with the agent over JSON-RPC. A2A deliberately standardizes only the communication format and leaves the question of *how clients discover a suitable agent* to the application. This is the gap that a marketplace must fill.

2.2 x402 Protocol

x402 [7] is a micropayment protocol that revives the HTTP 402 "Payment Required" status code. When a client requests a paid resource, the server responds with `402` together with machine-readable *payment requirements* (amount, asset, recipient, and network). The client then constructs a payment and retries the request with an `X-PAYMENT` header; the server verifies and settles the payment, typically through a *facilitator* that submits the transaction on chain, and finally returns the resource. Payments are denominated in stablecoins such as USDC and are made gasless for the payer using EIP-3009 *transfer with authorization* [4], which lets a user authorize a transfer with an off-chain signature that a third party submits. We rely on this property in Section 4.3 to delegate payment to the server without giving it custody of user funds.

2.3 On-Chain Agent Identity (ERC-8004)

ERC-8004 [8] standardizes a registry of agent identities on the Ethereum Virtual Machine. Built on the ERC-721 non-fungible token standard, it assigns each agent a token whose metadata (the Agent Card) is stored off chain and referenced by its content identifier. Because the registry is a public contract, any client can enumerate agents and resolve their metadata without trusting a central catalog, which makes it a natural *single source of truth* for a decentralized marketplace. A companion staking mechanism lets agents lock a deposit, giving them "skin in the game."

2.4 LLM-Based Agent Orchestration

The ReAct pattern [12] interleaves reasoning and acting: the LLM alternates between producing a thought, calling a tool, and observing the result, accumulating context until the task is done. This loop is a good fit for orchestration, because the set of agents, prices, and intermediate results is not known in advance and must be explored at run time. UniAgent implements the orchestration layer as a ReAct agent built with LangChain, exposing exactly two tools—one for discovery and one for execution—so that the model can operate the marketplace autonomously.

3 Related Work

Agent communication protocols. Several protocols now compete to standardize how agents interact. Ehtesham et al. [1] survey this space and compare the Model Context Protocol (MCP), which connects a single agent to tools and data sources, with peer-to-peer protocols such as A2A and the Agent Network Protocol (ANP). UniAgent adopts A2A for inter-agent communication because it is designed for cross-organization delegation, and uses an MCP-style tool interface only *inside* the orchestrator. None of these communication protocols specifies discovery or payment, which is precisely what our marketplace layer adds.

Agent payment systems. x402 [7] attaches stablecoin transfers to ordinary HTTP requests and settles them on a smart-contract chain. We build on x402 because it is open, HTTP-native, and requires no proprietary intermediary, and we extend it with delegated signatures and a server-side budget policy (Section 4.3).

Decentralized agent identity and closest prior work. Vaziry et al. [10] represent each agent's card as an individual smart contract

Table 1: UniAgent compared with Vaziry et al. [10].

Aspect	Vaziry et al.	UniAgent
Positioning	Protocol-level feasibility study	Marketplace architecture integrating the protocols
Identity	Per-agent smart contracts	Single ERC-8004 standard registry
Discovery	Four methods proposed, not integrated	Registry plus webhook-synced cache
Payment	x402 over A2A, demonstrated	x402 with delegated signing and budget policy
Orchestration	None (fixed agent $A \rightarrow B$)	LLM ReAct agent discovers, selects, executes
User experience	Protocol-level proof of concept	Chat UI, streaming, budget approval

and show that A2A, x402, and on-chain identity can interoperate. UniAgent instead consolidates identities into a single standardized ERC-8004 registry [8] rather than per-agent contracts, and—more importantly—builds a full marketplace and orchestration layer on top, as detailed below.

Difference from Vaziry et al. Table 1 summarizes the contrast. Vaziry et al. establish the technical *feasibility* of the protocol combination through one-to-one agent interactions. UniAgent treats those protocols as building blocks and contributes the missing layer above them: an LLM orchestrator that, given a single user task, autonomously discovers agents from a standardized registry, selects among them, pays them over x402, and returns an integrated result—together with the user-facing experience (chat, streaming, and budget approval) that this requires.

LLM multi-agent orchestration. AutoGen [11] shows how multiple LLM-based agents can coordinate through conversation, tool use, and optional human input within an application. UniAgent addresses a different layer: it treats the marketplace itself—open registration, cross-vendor discovery, and protocol-native payment—as the object of orchestration.

Centralized marketplaces. Centralized agent stores such as the OpenAI GPT Store [6] offer registration, discovery, and billing as a smooth, integrated experience, but only within one vendor's platform: agents are listed, ranked, and charged according to platform-specific rules, and there is no portability across providers. This resembles the broader platform pattern of walled gardens, where openness and neutrality are limited by the platform operator [3]. UniAgent aims to reproduce the convenience of such platforms while keeping each function open and vendor-neutral.

4 System Architecture

UniAgent is organized into three layers, shown together in Figure 1: a *marketplace layer* that maintains the catalog of available agents, an *orchestration layer* that turns a user task into a sequence of agent invocations, and a *payment layer* that settles each invocation. The layers are deliberately decoupled—the orchestrator does not need to know how identity is stored, and the payment layer does not need to know why an agent was selected—so that each can evolve independently. We describe each layer in turn and then explain how they combine into an end-to-end flow.

4.1 Marketplace Layer

The marketplace layer answers a single question: which agents exist and what can they do? Its authoritative source is the on-chain ERC-8004 `AgentIdentityRegistry`. Reading directly from the chain on every request, however, would be slow and costly, so UniAgent maintains an off-chain cache and keeps it consistent with the chain through an event-driven pipeline.

Figure 2 shows the developer-facing flow. An agent developer first deploys an A2A agent and uploads its Agent Card to IPFS (Phase A). The developer then registers the agent in the ERC-8004 registry, recording the IPFS content identifier as the token's metadata URI, and optionally stakes a deposit (Phase B). Registration and staking emit on-chain events. A blockchain indexing service (Alchemy) watches for these events and calls a webhook on the web application (Phase C); the webhook verifies the event, fetches the metadata from IPFS, and updates a PostgreSQL cache. Finally, the marketplace UI reads from this cache to list agents quickly and cheaply (Phase D). Because the chain remains the single source of truth and the cache is rebuilt from on-chain events, the cache can be regenerated at any time without loss of authority.

4.2 Orchestration Layer

The orchestration layer is a ReAct agent [12] built with LangChain and driven by a commercial LLM. It receives the user's natural-language task and operates the marketplace through two tools:

- `discover_agents(category, query)` returns a short list of candidate agents for a given need, drawn from the marketplace cache.
- `execute_agent(agentId, input)` invokes a chosen agent over A2A and pays for it over x402, returning the agent's result.

Given a task such as planning a trip, the agent reasons about which sub-tasks are needed, calls `discover_agents` for each, inspects the candidates, selects one, and calls `execute_agent`; it then observes the result and decides whether further steps are required. This loop continues until the task is complete, at which point the agent composes a final answer. Crucially, the model is never given a fixed plan or a hard-coded list of agents—both the decomposition and the choice of providers emerge from the ReAct loop at run time.

Discovery ranking. To keep the candidate list short and useful, `discover_agents` does not return every matching agent. Instead, the marketplace ranks candidates from the cache using a composite score that combines an agent's reliability and quality (estimated from past attestations), the size of its on-chain deposit, and a small bonus for newly registered agents, and returns the top three. The reliability and quality estimates are smoothed toward a global average so that agents with few reviews are neither unfairly promoted nor permanently excluded (mitigating the cold-start problem), and a small exploration term occasionally surfaces a new agent so that it can accumulate a track record. We describe the ranking qualitatively here, as a way to give the LLM a manageable and balanced

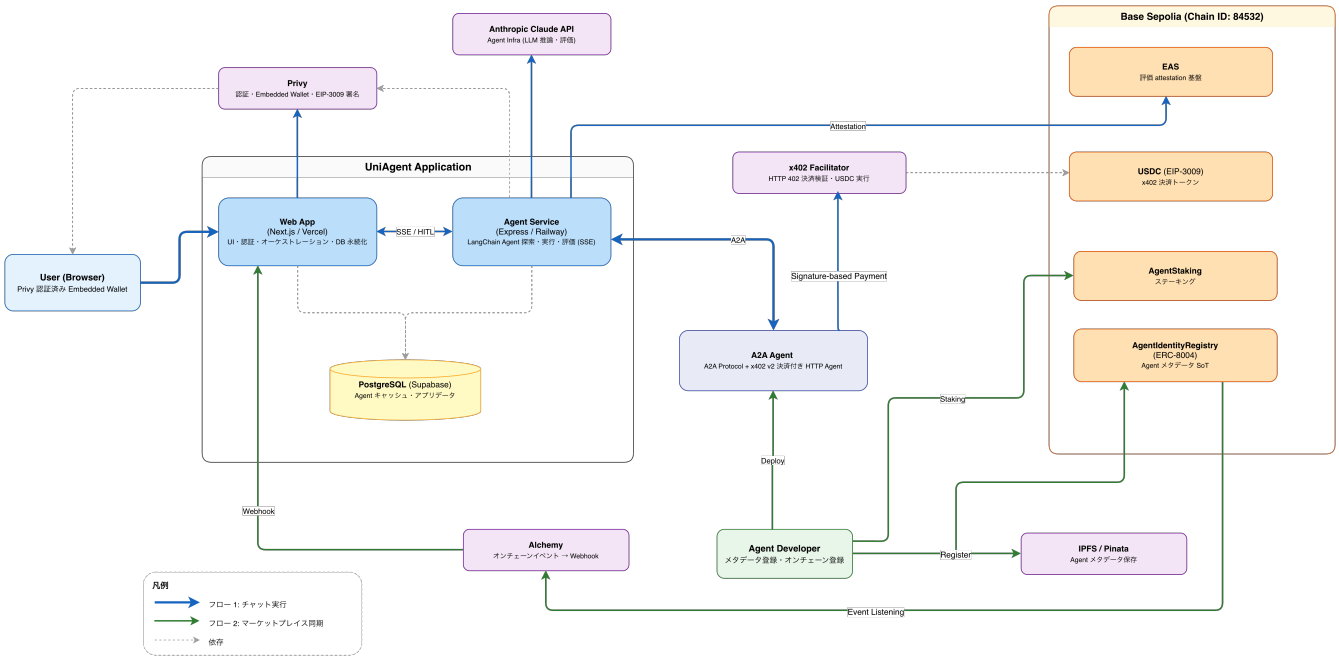

Figure 1: Overview of the UniAgent architecture. A user issues a natural-language task through the web application; an LLM-based agent service discovers candidate agents from an ERC-8004 registry (cached in PostgreSQL via Alchemy webhooks), invokes the selected external A2A agents, and settles payment over x402 using USDC on Base Sepolia.

set of options; its exact form is an engineering parameter rather than a contribution of this paper.

Budget control. Because the orchestrator can spend real funds, spending is governed by a server-side budget policy. Each user has an `autoApproveThreshold` stored in the database; payments below it proceed automatically, while payments above it pause the loop and request explicit user approval through the UI, following human-in-the-loop patterns common in multi-agent LLM frameworks [11]. This threshold is always read on the server and is never taken from the client, so a manipulated client cannot raise its own spending limit.

4.3 Payment Layer

The payment layer settles each `execute_agent` call using x402 [7]. When the orchestrator posts a task to an external agent's endpoint, the agent replies with HTTP 402 and its payment requirements. The orchestrator then creates an EIP-3009 *transfer with authorization* [4] for the requested amount of USDC and retries the request with the signed authorization in the `X-PAYMENT` header. The external agent forwards the payment to an x402 facilitator, which settles it on Base Sepolia, and then returns the result. We follow the standard x402 settlement sequence and refer the reader to the x402 specification for the detailed message diagram rather than reproducing it here.

UniAgent extends this baseline in two ways. First, signing is *delegated*: through Privy, the user authorizes the server to sign EIP-3009 transfers on their behalf, so payments are both gasless and non-custodial—the server can move funds within the user's policy but never holds the user's private key. Second, payment is bounded by the server-side budget policy described above, so delegation never becomes unbounded authority. After a successful execution, the system records an off-chain attestation of the interaction's quality and reliability using the Ethereum Attestation Service (EAS) [2]; these attestations feed back into the discovery ranking of Section 4.2, closing the loop between payment and reputation. A full evaluation of this reputation feedback is left to future work.

4.4 Integration Design

Figure 3 shows how the layers combine into a single end-to-end experience. After the user authenticates with Privy and submits a task, the web application opens a server-sent events (SSE) stream and forwards the task to the orchestrator. The orchestrator runs the ReAct loop—reasoning about the task, discovering agents (marketplace layer), executing the chosen agent with x402 payment (payment layer), and, when a payment exceeds the budget threshold, pausing for human-in-the-loop approval. Throughout, intermediate events—reasoning steps, tool calls, payments, and partial results—are streamed back to the UI over SSE, so the user sees the task unfold in real time rather than waiting for a single opaque response.

The value of the design comes from the combination rather than any single protocol. A2A makes agents callable across organizations; ERC-8004 makes them discoverable and identifiable without a central directory; x402 makes them payable with a single signed request; and the orchestration layer ties these together so that a user expresses a goal in natural language and the system carries out the underlying discovery, delegation, and payment automatically.

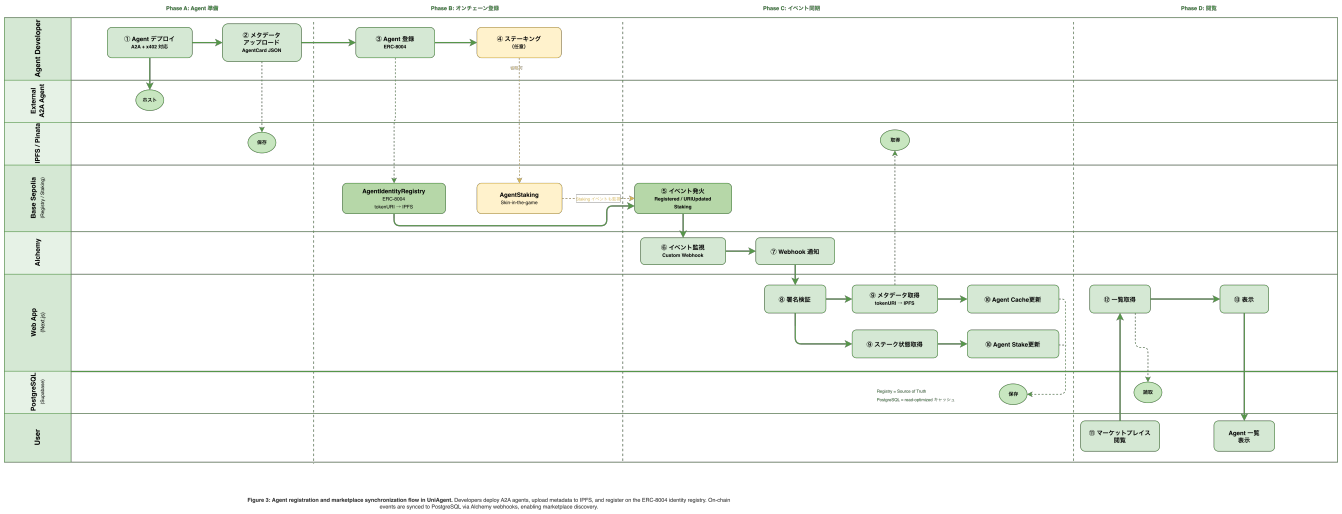

Figure 3: Agent registration and marketplace synchronization flow in UniAgent. Developers deploy A2A agents, upload metadata to IPFS, and register on the ERC-8004 identity registry. On-chain events are synced to PostgreSQL via Alchemy webhooks, enabling marketplace discovery.

Figure 2: Agent registration and marketplace synchronization. A developer deploys an A2A agent, uploads its Agent Card to IPFS, and registers it in the ERC-8004 registry (optionally staking a deposit). On-chain events are delivered to a webhook through a blockchain indexer, which refreshes the PostgreSQL cache that backs the marketplace UI.

5 Implementation

We implemented UniAgent as a TypeScript monorepo. The web application uses Next.js and React, with Privy for social login and delegated wallets. The orchestration service is a separate process built on LangChain with a commercial LLM, exposing the two tools described above and streaming results to the web app over SSE. The marketplace cache is stored in PostgreSQL and accessed through an ORM, with all database access confined to a dedicated data-access layer. Smart contracts are written in Solidity and developed with Hardhat. Payments use the x402 protocol with USDC on the Base Sepolia testnet.

Deployment. The ERC-8004 `AgentIdentityRegistry` is deployed on Base Sepolia (chain ID 84532) at `0x864A0C054AA6E9DBcCDB36a44a14A5A7bc81EB92`, and payments are settled in the testnet USDC token at `0x036CbD53842c5426634e7929541eC2318f3dCF7e`, which implements EIP-3009. An EAS schema is registered to record per-interaction quality and reliability attestations.

Demonstration scenario. For the prototype we focus on a travel-planning use case with two agent categories, *flight* and *hotel*, served as external A2A agents. A request such as "plan a three-day trip to Paris" is decomposed by the orchestrator into a flight sub-task and a hotel sub-task, each resolved through discovery and paid execution, and the results are combined into a single itinerary. The agents are deliberately operated as independent A2A services so that the flow exercises real cross-service discovery, communication, and payment rather than in-process calls.

Figures 4–6 illustrate the user interface. Figure 4 shows the chat view, where the user enters a task in natural language and watches reasoning steps, tool calls, and streamed responses appear over SSE. Figure 5 shows the marketplace view, which lists registered agents together with their on-chain ERC-8004 metadata. Figure 6 shows the budget-approval dialog that appears when a payment exceeds the user's `autoApproveThreshold`.

6 Evaluation

Our evaluation asks whether the integrated architecture can carry a realistic user task through discovery, delegation, and payment end to end, and at what cost. *The quantitative results in this section are placeholders to be replaced with measured values (marked* `XX`*).*

6.1 Experimental Setup

We deploy UniAgent on the Base Sepolia testnet with two categories of external A2A agents, flight and hotel. We define a task set spanning three classes: single-agent tasks, multi-agent tasks that require chaining two agents, and error-handling tasks designed to trigger fallback behavior. For each task we measure the end-to-end success rate, the wall-clock completion time, and the on-chain payment cost.

6.2 Scenarios and End-to-End Success Rate

We use four scenarios:

- **S1 (single agent):** "search for a hotel in Tokyo" should be resolved by a single hotel agent.
- **S2 (multi-agent):** "plan a three-day trip to Paris" should be decomposed and resolved by a flight agent and a hotel agent in sequence.
- **S3 (budget approval):** a task whose cost exceeds the threshold should pause and require user approval (cf. Figure 6).
- **S4 (agent failure):** when a selected agent does not respond, the orchestrator should fall back to an alternative.

Table 2 reports the success rate per scenario over $N = \texttt{XX}$ runs each. We also classify failures into LLM selection errors, A2A communication errors, and x402 payment timeouts.

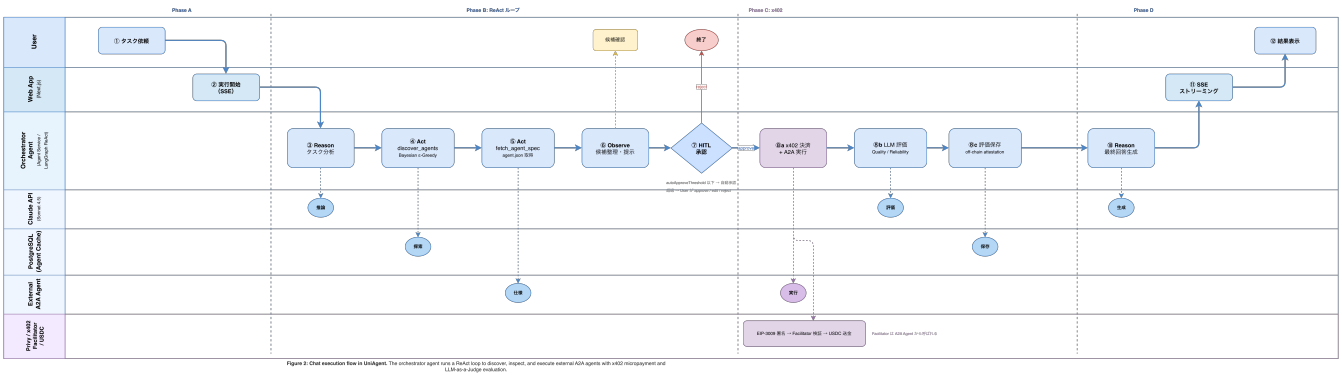

Figure 3: End-to-end execution flow. The orchestrator runs a ReAct loop that discovers, inspects, and executes external A2A agents, settling each call over x402; payments above the user's budget threshold trigger a human-in-the-loop approval. Progress is streamed to the UI over SSE.

Placeholder: chat UI screenshot (natural-language input, tool-call log, SSE streaming response). To be added.

Figure 4: Chat UI: natural-language task input with streamed reasoning steps, tool calls, and responses.

Placeholder: marketplace screenshot (list of registered agents with on-chain ERC-8004 metadata). To be added.

Figure 5: Marketplace UI: registered agents and their on-chain ERC-8004 metadata.

Table 2: End-to-end success rate by scenario (placeholder values).

Scenario	Type	Runs	Success rate
S1	single agent	XX	XX%
S2	multi-agent	XX	XX%
S3	budget approval	XX	XX%
S4	agent failure	XX	XX%

Placeholder: budget-approval screenshot (approval dialog shown when a payment exceeds the threshold). To be added.

Figure 6: Budget-approval UI: the dialog shown when a payment exceeds the user's `autoApproveThreshold`.

Table 3: Payment latency and cost per task (placeholder values).

Metric	Value
x402 settlement latency (ms)	XX
Gas per task (gas units)	XX
End-to-end task time (s)	XX

6.3 Payment Cost and Latency

Table 3 reports the additional latency introduced by the x402 payment step and the on-chain gas cost per task as measured on Base Sepolia. The payment overhead is expected to be small relative to the LLM reasoning and external-agent execution time.

6.4 Comparison with Existing Approaches

Table 4 compares UniAgent feature-by-feature with the protocol-level study of Vaziry et al. [10] and with a centralized platform such as the GPT Store [6]. UniAgent matches the end-to-end convenience of the centralized platform while keeping registration, discovery, and payment open and vendor-neutral, and it adds the orchestration layer that the protocol-level study lacks. The centralized platform achieves an end-to-end flow only within a closed environment, whereas UniAgent does so across independent providers.

Table 4: Feature comparison.

Feature	Vaziry et al.	UniAgent	Centralized (GPT Store)
Registration	Custom contract	ERC-8004	Platform-specific
Discovery	Proposed, not integrated	Registry + cache	In-platform search
Payment	x402 PoC	x402 + budget	Platform billing
Orchestration	None	LLM ReAct	Platform-dependent
End-to-end flow	No	Yes	Yes (closed)
Vendor-neutral	Yes	Yes	No

7 Discussion

The prototype shows that an open, integrated marketplace is feasible: once registration, discovery, communication, and payment are each expressed as open protocols, an LLM orchestrator can stitch them into an autonomous end-to-end flow. We expect the dominant cost to be LLM reasoning and external-agent execution rather than payment, since x402 settlement adds only a single signed request and one on-chain transfer.

UniAgent has several limitations. It currently runs on a testnet, so real economic incentives and adversarial behavior are not yet exercised. The evaluation uses two agent categories (flight and hotel); broader domains and a larger agent population remain to be tested. The discovery ranking and the EAS-based reputation feedback are presented here as design elements and have not been validated at scale. Finally, end-to-end quality depends on the LLM's selection accuracy: prior benchmarks show that reasoning, decision-making, and instruction following remain key failure modes for LLM agents [5], so a poor choice among candidates can lower task success even when every protocol behaves correctly.

Future work includes large-scale evaluation of the discovery and reputation mechanisms, hardening the system for mainnet (security and key management for delegated payments), refining the economic model for staking and pricing, and extending orchestration to richer multi-agent workflows.

8 Conclusion

We presented UniAgent, a decentralized agent marketplace that integrates the A2A communication protocol, x402 micropayments, and ERC-8004 on-chain identity under an LLM-based orchestration layer. Whereas prior work demonstrated these protocols in isolation or as one-to-one interactions, and centralized platforms offer end-to-end convenience only within a closed ecosystem, UniAgent shows how to combine open protocols into a coherent, vendor-neutral architecture in which a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. A working prototype on Base Sepolia demonstrates the full flow over independent flight- and hotel-booking agents. We believe this integrated design is a step toward an open "web of agents" in which users delegate tasks across organizational boundaries as easily as they browse the web today.

Acknowledgments

References

[1] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). https://arxiv.org/abs/2505.02279. arXiv:2505.02279. Accessed: 2026-05-31.

[2] Ethereum Attestation Service. 2024. Ethereum Attestation Service (EAS) Documentation. https://docs.attest.org/. Accessed: 2026-05-31.

[3] Rob Frieden. 2016. The Internet of Platforms and Walled Gardens: Implications for Openness and Neutrality. SSRN Working Paper. SSRN:2754583. Accessed: 2026-05-31.

[4] Peter Jihoon Kim, Kevin Britz, and David Knott. 2020. ERC-3009: Transfer With Authorization. https://eips.ethereum.org/EIPS/eip-3009. Ethereum Improvement Proposals, no. 3009, September 2020. Accessed: 2026-05-31.

[5] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. arXiv:2308.03688.

[6] OpenAI. 2024. Introducing the GPT Store. https://openai.com/index/introducing-the-gpt-store/. Accessed: 2026-05-31.

[7] Erik Reppel and Coinbase. 2025. x402: An Open Standard for Internet-Native Payments. https://www.x402.org/x402-whitepaper.pdf. Coinbase Developer Platform whitepaper. Accessed: 2026-05-31.

[8] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. 2025. ERC-8004: Trustless Agents. https://eips.ethereum.org/EIPS/eip-8004. Ethereum Improvement Proposals, no. 8004, August 2025. Accessed: 2026-05-31.

[9] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. Agent2Agent (A2A) Protocol. https://a2a-protocol.org/. Specification, Google. Accessed: 2026-05-31.

[10] Awid Vaziry, Sandro Rodriguez Garzon, and Axel Küpper. 2025. Towards Multi-Agent Economies: Enhancing the A2A Protocol with Ledger-Anchored Identities and x402 Micropayments for AI Agents. https://arxiv.org/abs/2507.19550. arXiv:2507.19550 [cs.MA]. Accessed: 2026-05-31.

[11] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. https://arxiv.org/abs/2308.08155. arXiv:2308.08155. Accessed: 2026-05-31.

[12] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. arXiv:2210.03629.